AI-ASSISTED CODING
Assignment - 16.1

HT NO :2303A510A2
Batch – 14
NAME : M BHARATH

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

TASK 1 – Student Information System Schema

Prompt:

Design a relational database schema for a Student Information System.
Requirements:

- Tables: Students, Courses, Enrollments
- Define primary keys and foreign keys
- Ensure proper relationships (many-to-many between students and courses)

Generate:

- SQL CREATE TABLE statements
- SQL query to insert a new student record
- SQL query to fetch all courses enrolled by a specific student
- SQL query to count number of students in each course Ensure

the schema is normalized (up to 3NF).

Code:

```sql
-- STUDENT INFORMATION SYSTEM
-- Drop tables if already exist
(optional) DROP TABLE IF EXISTS
Enrollments;
DROP TABLE IF EXISTS
Students; DROP TABLE IF
EXISTS Courses;
-- Create Tables
CREATE TABLE Students (
   student_id INTEGER PRIMARY KEY AUTOINCREMENT,
   first_name TEXT NOT
   NULL, last_name TEXT
   NOT NULL, email TEXT
   UNIQUE NOT NULL
);
CREATE TABLE Courses (
   course_id INTEGER PRIMARY KEY AUTOINCREMENT,
   course_name TEXT NOT NULL,
   course_code TEXT UNIQUE
   NOT NULL
);
CREATE TABLE Enrollments (
   enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
   student_id INTEGER,
   course_id INTEGER,
   FOREIGN KEY (student_id) REFERENCES
   Students(student_id), FOREIGN KEY
   (course_id) REFERENCES Courses(course_id)
);
-- Insert Sample Students
INSERT INTO Students (first_name, last_name,
email) VALUES ('John', 'Doe',
```

```sql
'john.doe@example.com'),
('Alice', 'Smith',
'alice.smith@example.com'), ('Bob',
'Johnson',
'bob.johnson@example.com');
-- Insert Sample Courses
INSERT INTO Courses (course_name,
course_code) VALUES ('Database
Management', 'DBMS101'),
('Computer Networks',
'CN102'), ('Operating
Systems', 'OS103'),
('Data Structures',
'DS104');
-- Insert Enrollments
INSERT INTO Enrollments (student_id,
course_id) VALUES (1, 1),
```

```sql
(1, 2),
(2, 1),
(2, 3),
(3, 2),
(3, 4);
-- Query 1: Fetch all courses of a student
SELECT s.first_name, s.last_name, c.course_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON c.course_id = e.course_id
WHERE s.email = 'john.doe@example.com';
-- Query 2: Count students in each course
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

Output:

SQL ▼ 1 / 1 1 - 2 of 2

first_name	last_name	course_name
John	Doe	Database Management
John	Doe	Computer Networks

SQL ▼ 1 / 1 1 - 4 of 4

course_name	student_count
Computer Networks	2
Data Structures	1
Database Management	2
Operating Systems	1

TASK 2 – Hospital Management Database

Prompt:

Design a relational database schema for a Hospital Management System.
Requirements:

- Tables: Doctors, Patients, Appointments
- Include constraints like primary keys, foreign keys, unique IDs, and valid date fields
- Ensure normalization (up to 3NF)

Generate:

- SQL CREATE TABLE statements
- Query to list all appointments for a specific doctor
- Query to retrieve patient history by patient ID
- Query to count total patients treated by each doctor Use

JOIN operations where necessary.

Code:

```
-- HOSPITAL MANAGEMENT SYSTEM
-- Drop tables if already exist
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Patients;
DROP TABLE IF EXISTS Doctors;
-- Create Tables
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_name TEXT NOT NULL,
    specialization TEXT NOT NULL,
    email TEXT UNIQUE
);
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    patient_name TEXT NOT NULL,
```

```
    age INTEGER CHECK(age > 0),
    gender TEXT,
    phone TEXT
    UNIQUE
);
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY
    AUTOINCREMENT, doctor_id INTEGER,
    patient_id INTEGER,
    appointment_date TEXT NOT
    NULL, diagnosis TEXT,
    FOREIGN KEY (doctor_id) REFERENCES
    Doctors(doctor_id), FOREIGN KEY
    (patient_id) REFERENCES
    Patients(patient_id)
);
-- Insert Sample Doctors
INSERT INTO Doctors (doctor_name,
specialization, email) VALUES ('Dr. Rajesh
Kumar', 'Cardiologist', 'rajesh@hospital.com'),
('Dr. Priya Sharma', 'Neurologist',
'priya@hospital.com'), ('Dr. Amit Verma',
'Orthopedic', 'amit@hospital.com');
-- Insert Sample Patients
INSERT INTO Patients (patient_name, age, gender,
phone) VALUES ('John Doe', 25, 'Male',
'9876543210'),
('Alice Smith', 30, 'Female', '9876543211'),
('Bob Johnson', 40, 'Male', '9876543212');
-- Insert Appointments
INSERT INTO Appointments (doctor_id, patient_id,
appointment_date, diagnosis) VALUES
(1 1, '2026-03-  'Heart
 ,       01',    Checkup'),
```

```sql
(1, 2, '2026-03-02', 'Chest Pain'),
(2, 1, '2026-03-03', 'Migraine'),
(3, 3, '2026-03-04', 'Fracture'),
(2, 2, '2026-03-05', 'Headache');
```

```sql
-- Query 1: List all appointments for a specific doctor
-- (Example: Doctor ID = 1)
SELECT d.doctor_name, p.patient_name,
a.appointment_date, a.diagnosis FROM Appointments a
JOIN Doctors d ON a.doctor_id =
d.doctor_id JOIN Patients p ON
a.patient_id = p.patient_id WHERE
d.doctor_id = 1;
-- Query 2: Retrieve patient history by patient ID
-- (Example: Patient ID = 1)
```

```sql
SELECT p.patient_name, d.doctor_name, a.appointment_date, a.diagnosis
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE p.patient_id = 1;
-- Query 3: Count total patients treated by each doctor
SELECT d.doctor_name, COUNT(DISTINCT a.patient_id) AS total_patients
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_name;
```

Output:

SQL ▼ 1 / 1 1 - 2 of 2

doctor_name	patient_name	appointment_date	diagnosis
Dr. Rajesh Kumar	John Doe	2026-03-01	Heart Checkup
Dr. Rajesh Kumar	Alice Smith	2026-03-02	Chest Pain

SQL ▼ 1 / 1 1 - 2 of 2

patient_name	doctor_name	appointment_date	diagnosis
John Doe	Dr. Rajesh Kumar	2026-03-01	Heart Checkup
John Doe	Dr. Priya Sharma	2026-03-03	Migraine

SQL ▼ 1 / 1 1 - 3 of 3

doctor_name	total_patients
Dr. Amit Verma	1
Dr. Priya Sharma	2
Dr. Rajesh Kumar	2

TASK 3 – Library Database

Prompt:

Design a relational database schema for a Library Management System.
Requirements:

- Tables: Books, Members, Loans
- Define primary and foreign keys
- Ensure normalization (up to 3NF)
- Suggest indexing strategies for faster query performance

Generate:

- SQL CREATE TABLE statements
- Query to retrieve all books currently issued
- Query to find overdue books (loan date > 30 days)
- Query to count number of books loaned by each member Include

JOINs and WHERE conditions.

Code:

```
DROP TABLE IF EXISTS Loans;
DROP TABLE IF EXISTS Members;
DROP TABLE IF EXISTS Books;
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    isbn TEXT UNIQUE,
    available_copies INTEGER NOT NULL CHECK(available_copies >= 0)
);
CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    member_name TEXT NOT NULL,
    email TEXT UNIQUE,
    phone TEXT UNIQUE
```

```sql
);
CREATE TABLE Loans (
   loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
   book_id INTEGER,
   member_id INTEGER,
   loan_date TEXT NOT
   NULL, return_date
   TEXT,
   FOREIGN KEY (book_id) REFERENCES
   Books(book_id), FOREIGN KEY (member_id)
   REFERENCES Members(member_id)
);
CREATE INDEX idx_loans_book_id ON
Loans(book_id); CREATE INDEX
idx_loans_member_id ON Loans(member_id);
CREATE INDEX idx_loans_loan_date ON
Loans(loan_date);
INSERT INTO Books (title, author, isbn,
available_copies) VALUES ('Database System
Concepts', 'Silberschatz', 'ISBN001', 5),
('Computer Networks', 'Andrew Tanenbaum',
'ISBN002', 3), ('Operating Systems', 'Galvin',
'ISBN003', 4),
('Data Structures', 'Seymour Lipschutz',
'ISBN004', 2); INSERT INTO Members
(member_name, email, phone) VALUES
('John Doe', 'john@example.com',
'9000000001'), ('Alice Smith',
'alice@example.com', '9000000002'),
('Bob Johnson', 'bob@example.com',
'9000000003');
INSERT INTO Loans (book_id, member_id, loan_date,
return_date) VALUES
```

```
(1, 1, '2026-02-20', NULL),
(2, 1, '2026-02-15', '2026-02-25'),
(3, 2, '2026-02-10', NULL),
(4, 3, '2026-01-01', NULL),
(2, 2, '2026-01-15', '2026-02-20');
```

```
SELECT b.title, m.member_name,
l.loan_date FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id =
m.member_id WHERE l.return_date
IS NULL;
SELECT b.title, m.member_name,
l.loan_date FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id =
m.member_id WHERE l.return_date
IS NULL
AND DATE(l.loan_date) <= DATE('now', '-
30 days'); SELECT m.member_name,
COUNT(l.loan_id) AS total_books
```

```sql
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_name;
```

Output:

SQL ▼ 1 / 1 1 - 3 of 3

title	member_name	loan_date
Database System Concepts	John Doe	2026-02-20
Operating Systems	Alice Smith	2026-02-10
Data Structures	Bob Johnson	2026-01-01

SQL ▼ 1 / 1 1 - 3 of 3

title	member_name	loan_date
Database System Concepts	John Doe	2026-02-20
Operating Systems	Alice Smith	2026-02-10
Data Structures	Bob Johnson	2026-01-01

SQL ▼ 1 / 1 1 - 3 of 3

member_name	total_books
Alice Smith	2
Bob Johnson	1
John Doe	2

TASK 4 – Real-Time Application: Online Shopping Database

Prompt:

Design a relational database schema for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails
- Define primary keys, foreign keys, and relationships
- Ensure normalization (up to 3NF)
- Suggest query optimization techniques

Generate:

- SQL CREATE TABLE statements
- Query to retrieve all orders by a user
- Query to find the most purchased product
- Query to calculate total revenue in a given month

Include aggregation functions, JOINs, and optimization suggestions.

Code:

```
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    phone TEXT UNIQUE
);
CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL CHECK(price > 0),
    stock INTEGER NOT NULL CHECK(stock >= 0)
);
CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    order_date TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

```sql
CREATE TABLE OrderDetails (
   order_detail_id INTEGER PRIMARY KEY
   AUTOINCREMENT, order_id INTEGER,
   product_id INTEGER,
   quantity INTEGER NOT NULL
   CHECK(quantity > 0), FOREIGN KEY
   (order_id) REFERENCES Orders(order_id),
   FOREIGN KEY (product_id) REFERENCES
   Products(product_id)
);
CREATE INDEX idx_orders_user_id ON Orders(user_id);
CREATE INDEX idx_orderdetails_order_id ON
OrderDetails(order_id); CREATE INDEX
idx_orderdetails_product_id ON
OrderDetails(product_id);
INSERT INTO Users (user_name, email,
phone) VALUES ('John Doe',
'john@example.com', '9000000001'),
('Alice Smith', 'alice@example.com',
'9000000002'), ('Bob Johnson',
'bob@example.com', '9000000003');
INSERT INTO Products (product_name, price,
stock) VALUES ('Laptop', 60000, 10),
('Mobile Phone', 20000, 20),
('Headphones', 2000, 50),
('Keyboard', 1500, 30);
INSERT INTO Orders (user_id,
order_date) VALUES (1, '2026-03-
01'),
(1, '2026-03-05'),
(2, '2026-03-10'),
(3, '2026-02-20');
INSERT INTO OrderDetails (order_id, product_id, quantity)
VALUES
```

```
(1 1, 1)
 ,     ,
(1 3, 2)
 ,     ,
(2 2, 1)
 ,     ,
(2 4, 1)
 ,     ,
(3 2, 2)
 ,     ,
(4 3, 3)
 ,     ;
```

```
SELECT u.user_name, o.order_id,
o.order_date FROM Users u
JOIN Orders o ON u.user_id =
o.user_id WHERE u.user_id =
1;
SELECT p.product_name, SUM(od.quantity) AS
total_quantity FROM OrderDetails od
JOIN Products p ON od.product_id =
p.product_id GROUP BY
p.product_name
```

```sql
ORDER BY total_quantity DESC
LIMIT 1;
SELECT SUM(p.price * od.quantity) AS total_revenue
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
JOIN Orders o ON od.order_id = o.order_id
WHERE strftime('%Y-%m', o.order_date) = '2026-03';
```

Output:

SQL ▼ 1 / 1 1 - 2 of 2

user_name	order_id	order_date
John Doe	1	2026-03-01
John Doe	2	2026-03-05

SQL ▼ 1 / 1 1 - 1 of 1

product_name	total_quantity
Headphones	5

SQL ▼ 1 / 1 1 - 1 of 1

total_revenue
125500.0

TASK 5 – University Examination Database

Prompt:

Design a relational database schema for a University Examination System.

Requirements:

- Tables: Students, Subjects, Exams, Results

- Define primary keys, foreign keys, and referential integrity constraints
- Ensure normalization (up to 3NF)

Generate:

- SQL CREATE TABLE statements
- Query to insert examination results for a student
- Query to retrieve subject-wise marks for a student
- Query to calculate average marks for each subject Use

JOINs and aggregate functions.

Code:

```
DROP TABLE IF EXISTS Results;
DROP TABLE IF EXISTS Exams;
DROP TABLE IF EXISTS Subjects;
DROP TABLE IF EXISTS Students;
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_name TEXT NOT NULL,
    email TEXT UNIQUE
);
CREATE TABLE Subjects (
    subject_id INTEGER PRIMARY KEY AUTOINCREMENT,
    subject_name TEXT NOT NULL,
    subject_code TEXT UNIQUE NOT NULL
);
CREATE TABLE Exams (
    exam_id INTEGER PRIMARY KEY AUTOINCREMENT,
    subject_id INTEGER,
    exam_date TEXT NOT NULL,
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
);
CREATE TABLE Results (
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    exam_id INTEGER,
    marks INTEGER CHECK(marks >= 0 AND marks <= 100),
```

```sql
    FOREIGN KEY (student_id) REFERENCES
    Students(student_id), FOREIGN KEY
    (exam_id) REFERENCES Exams(exam_id)
);
INSERT INTO Students (student_name,
email) VALUES ('John Doe',
'john@example.com'),
('Alice Smith',
'alice@example.com'), ('Bob
Johnson', 'bob@example.com');
INSERT INTO Subjects (subject_name,
subject_code) VALUES ('Mathematics',
'MATH101'),
('Physics', 'PHY102'),
('Computer Science', 'CS103');
INSERT INTO Exams (subject_id,
exam_date) VALUES (1, '2026-03-01'),
(2, '2026-03-02'),
(3, '2026-03-03');
INSERT INTO Results (student_id, exam_id, marks) VALUES
(1, 1, 85),
(1, 2, 78),
(1, 3, 90),
(2, 1, 88),
(2, 2, 76),
(3, 3, 92);
SELECT * FROM Results;
SELECT s.student_name,
sub.subject_name, r.marks FROM
Results r
```

```sql
JOIN Students s ON r.student_id =
s.student_id JOIN Exams e ON
r.exam_id = e.exam_id
JOIN Subjects sub ON e.subject_id =
sub.subject_id WHERE s.student_id = 1;
SELECT sub.subject_name, AVG(r.marks) AS
average_marks FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects sub ON e.subject_id =
sub.subject_id GROUP BY
sub.subject_name;
```

Output:

SQL ▼ 1 / 1 1 - 6 of 6

result_id	student_id	exam_id	marks
1	1	1	85
2	1	2	78
3	1	3	90
4	2	1	88
5	2	2	76
6	3	3	92

SQL ▼ 1 / 1 1 - 3 of 3

student_name	subject_name	marks
John Doe	Mathematics	85
John Doe	Physics	78
John Doe	Computer Science	90

SQL ▼ 1 / 1 1 - 3 of 3

subject_name	average_marks
Computer Science	91.0
Mathematics	86.5
Physics	77.0